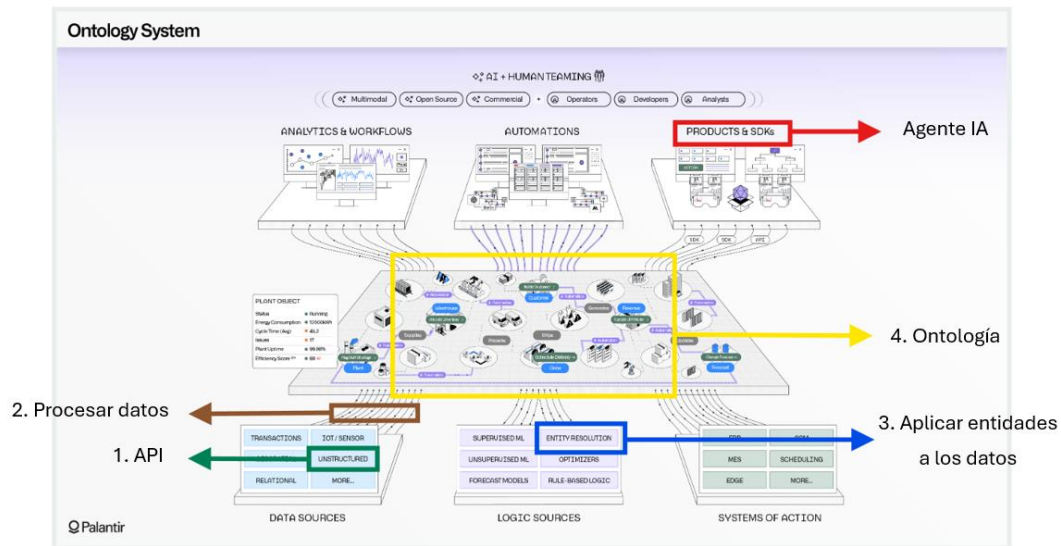


ARQUITECTURA



1. API: Extrae ids de normas consolidadas e itera una a una guardándolas enteras
2. Se preprocesan los datos crudos y se extrae la información relevante a guardar
3. Se almacena la información relevante en esquemas fijos. Para este caso en Neo4J, una base de datos no estructurados para grafos.
4. Los esquemas fijos son los que definen la capa semántica de la ontología. Para este caso:
 - a. Esquemas de la capa kinética (capa de ingesta de datos):
 - i. Nodos: Norma
 - ii. Aristas: MODIFICA, DEROGA, CITA (solo una dirección, sin duplicación)
 - b. Esquemas de la capa dinámica (capa en la que escriben modelos ML):
 - i. Nodos: UserQuery
 - ii. Aristas: ResultEdge
5. LLM que recibe una query del usuario, procesa lo que necesita, accede a los esquemas de datos y ejecuta una tool que llama a la BBDD y ésta devuelve el resultado.
 - a. Con la query del usuario, la respuesta, y la consulta a la BBDD se crea un nodo UserQuery. Y si la respuesta menciona algún nodo Norma une UserQuery a los nodos Norma mediante ResultEdge.
6. Frontend: web con acceso al chat y al grafo, en el que se pueden hacer operaciones de filtrado.

Por qué este diseño: es un diseño bastante simple, ajustado a la tarea. Sin procesamiento de texto semántico, ni procesamiento en paralelo (API y preprocesado). Lo valoré inicialmente pero no merece la pena la complejidad para el caso. De los datos extraídos se almacenan solo metadatos y referencias anteriores y posteriores, todo se extrae y procesa secuencialmente, ya que son operaciones que se harán una vez masivamente. No merece la pena tampoco separar en distintos nodos información (para el caso del texto, si, pero no aplica). En cambio, para las aristas si que merece la pena, cada una es una acción distinta. Para el LLM he escogido el modelo más rápido y barato de Claude (Haiku), por simplicidad y más que suficiente para la tarea, se usa con temperatura=0 para garantizar determinismo en las respuestas, importante para el sector.

